Semester 2 Lab 11: Android App Development

Introduction

In this lab, we will follow two online official tutorials from codelabs.developers.google.com. The first one will help us understand how to manage activities and intents. The second one will help us understand the lifecycle of Android activities.

Suppose you have not completed the Android Studio lab from last week. In that case, it is strongly recommended that you do so before attempting this lab because familiarity with Android Studio is assumed here.

Activities and Intents

As we learned in the lecture, an `Activity` represents a single screen in your app that provides a single functionality, usually in a full-screen window.

Navigation between activities happens by means of **intents**. An `Intent` is an asynchronous action request that starts one activity from another activity, possibly passing some data.

When a new activity starts, the previous one is stopped and put onto the "back stack" (*last in, first out* stack). This mechanism allows the system to resume the previous activity when the user is finished with the new one (e.g., pressing the Back button).

Intents can be *explicit* or *implicit*:

- An *explicit intent* starts a specific activity (using its fully qualified class name).
- An *implicit intent* indicates a general action to perform instead of a specific activity.

In this lab, we will create two activities (a parent named `MainActivity` and a child named `SecondActivity`), and we will use an explicit intent to navigate and pass data between the two activities.

Create Android App Project

Create a new Android app project with an empty view activity to begin with. Name your project `Lab11` and use package name `uk.edu.le.co2124.lab11`. Use API 26 ("Oreo"; Android 8.0) as the minimum SDK. Your first activity will be called `MainActivity,` and its XML layout is `activity_main.xml`.

Create a second activity named `SecondActivity` by right-clicking on the *app* item in your project explorer and choosing `New > Activity > Empty Activity`.

MainActivity

Open the activity_main.xml under res and add a button with id `buttonSend` and text **Send** (use resource `@string/button_send="Send"` instead of hardcoding the string `"Send"`), you can declare this within the strings.xml under values (`<string`

name="send_button">Send</string>). When the user clicks this button, your main activity will use an **explicit intent** to start the second activity. For this, set the *onClick* property of *MainActivity* to *openSecondActivity* (to achieve this, open the activity_main.xml code and add the following to the buttons attributes android:onClick="openSecondActivity") and implement this method in *MainActivity.java*. Remember that you need a single argument in the method of type *View*. Make sure you also include a logging statement in the method. The code should look like this

```
public void openSecondActivity(View view){
    Log.d(TAG, "Button clicked");
    Intent intent = new Intent(this, SecondActivity.class);
    startActivity(intent);
}
```

Execute your application and test that the navigation works well.

Next, add *EditText* view (Plain Text) with id *editTextMessage* to the main activity. Clear the *text* property, and set the *hint* property to *"Write message here"* using a string resource *@string/message_hint* (<string name="message_hint">Write message here</string>). The user enters a message and clicks **Send**. Extend the code of your *openSecondActivity* method in the MainActivity.java to include the text of this *editTextMessage* in the intent. For this, you can follow these steps:

1. Add a global constant to *MainActivity*:

   ```
   public static final String EXTRA_MESSAGE = "uk.edu.le.co2124.lab11.extra.MESSAGE";
   ```

2. Create a variable *EditText mEditText* as a private field in the class.

3. In the *onCreate* method, use *findViewById* to retrieve the object associated to resource *R.id.editTextMessage* and assign the output to your *mEditText* variable. i.e. mEditText = findViewById(R.id.editTextMessage);

4. In *openSecondActivity*, **right before *startActivity(intent)***:

   1. Obtain the string from the *text* property of the *EditText* object and assign it to a string variable named *message* (String message = mEditText.getText().toString();)

   2. use the *putExtra* method of the intent to add the string variable as an *extra*:

      ```
      intent.putExtra(EXTRA_MESSAGE, message);
      ```

SecondActivity

Now, let's receive the intent in our second activity. Open the Layout Editor view for *activity_second.xml*.

Drag a *TextView* from the **Palette** and add constraints to the top and left sides of the layout (*16dp* margin to the top and *16dp* margin to the left). Set the id of the view to *text_message*.

Now, let's add the text received from the intent to the text view we have just added. For this, you will add some code to the *SecondActivity.onCreate()* method. First, get the intent that activated the creation of the second activity:

```
Intent intent = getIntent();
```

Then, get the message from the intent's extra:

```
String message = intent.getStringExtra(MainActivity.EXTRA_MESSAGE);
```

Use *findViewByID()* to get a reference to the *TextView* for the message from the layout *R.id.text_message* (`TextView textView = findViewById(R.id.text_message);`) and set its text (*setText*) to the string message received (`textView.setText(message);`).

Finally, ensure that *SecondActivity* declares *MainActivity* as its parent activity in *AndroidManifest.xml*.

In your manifest, the second activity should look like this

```
<activity
    android:name=".SecondActivity"
    android:exported="false"
    android:parentActivityName=".MainActivity">
    <meta-data
        android:name="android.support.PARENT_ACTIVITY"
        android:value="uk.edu.le.co2124.lab11.MainActivity" />
</activity>
```

Execute your app again and check that the message is being passed correctly to *SecondActivity* and that you have the **Up** button to go back to *MainActivity*.

The **Up button** is part of the **ActionBar** or **Toolbar**. If your app is using a theme (this can be found res – values -themes) that doesn't include an ActionBar (e.g., Theme.AppCompat.NoActionBar), the Up button won't appear. Ensure your app is using a theme that includes an ActionBar, such as Theme.AppCompat.Light.DarkActionBar.Returning Message to Calling Activity

So far, we have been passing messages one way, from our main activity to a second activity (in the previous lab, we attached the event handler in the code, and in this lab, we have attached an existing method to the *onClick* property of a *Button* view). Now, let's look into how to return messages back to the calling activity, which is often very useful when implementing real apps.

To accomplish this, we will again use an *Intent* and intent *extras*.

Add Reply *EditText* View

First, let's add an *EditText* view (Plain Text) to the layout of *SecondActivity*, add a constraint from the top of the new edit text to the bottom of the already existing text view

and from the left margin of the edit text to the left margin of the screen. Set its id to *editTextRespond*, clear its *text* property and set its *hint* property to the already created *message_hint* string resource. If Android Studio warns you that the view is missing the *autofillHints* attribute, you can simply add the following to the view's XML tag: *android:importantForAutofill="no"*.

Add *ImageButton* View

Now, drag an *ImageButton* object to your layout. This view is very similar to a regular button, except that you can set an image to it. When you drop the view on your layout, you will be prompted to choose an image for it; scroll in the list to find the drawable resource named *@android:drawable/ic_menu_send* (you will notice that it looks a lot like the WhatsApp send button).

Keep the name of this button view as *imageButton*. Add a constraint from its top to the bottom of *editTextRespond* and to the left margin of the screen. You will notice that Android Studio warns you that the image button does not have a *ContentDescription* i.e., imageButton <ImageButton>: No speakable text present. It is a very good idea to set this property, as it enables users with *accessibility requirements* to use your app (e.g., through a screen reader). We will fix this warning by setting the *ContentDescription* property to a new string resource named *respond_button*. Open *res/values/strings.xml*, add a new string resource directly to the XML (*<string name="respond_button">Respond Button</string>*) and set it as the *imageButton*'s *ContentDescription* property.

Sending Reply Back

Now, let's add the functionality to return a reply to *MainActivity*. From the XML code, set the *onClick* property of *imageButton* to *returnReply* (`android:onClick="returnReply"`). In the XML *Code* view of your *SecondActivity*. The text *returnReply* will be marked in red because the method does not exist. Android Studio can generate the code skeleton for you; just place your cursor on *returnReply* and press *Alt+Enter* (*Option+Return* on MacOS), and select the option ***Create 'returnReply(View)' in 'SecondActivity'.***

First, add the following global string constant and *EditText* variable to the top of *SecondActivity*:

```
public static final String EXTRA_REPLY = "uk.edu.le.co2124.lab11.extra.REPLY"
;
private EditText mReply;
```

In *SecondActivity.onCreate()*, in between lines *setContentView(...);* and *Intent intent = getIntent();*, get a reference to the *editTextRespond* view from the layout:

```
mReply = findViewById(R.id.editTextRespond);
```

Now, use the same idea that you used in *MainActivity.openSecondActivity* to create an *Intent* named *replyIntent* and add the text from *mReply* as an extra. This time, however, instead of then calling *starActivity(intent)*, you are going to write:

```
setResult(RESULT_OK, replyIntent);
finish();
```

What you are saying here is: This intent is meant to set the result of the current activity (*RESULT_OK* means the activity executed successfully, alternatively you could use *RESULT_CANCELLED* if something went wrong or the user cancelled the task) and terminate it. *finish()* will close the current activity and return to *MainActivity*.

To finish this task, go to the *MainActivity*, add a *TextView* object named *text_reply* and constrain it to the bottom of the *Send* button and centered horizontally. Set its *visibility* attribute to *invisible*, because we do not want it to be displayed when *MainActivity* is created and there is no reply yet.

Add a private field to hold a reference for the reply text view:

```
private TextView mReplyTextView;
```

Add the appropriate line of code at the end of the *onCreate* method to find view by id *R.id.text_reply* and assign it to *mReplyTextView*.

startActivityForResult

When you use an explicit *Intent* to start another *Activity* without expecting any response, you use *startActivity(intent)*, as we have done earlier in *openSecondActivity*. However, now we want to start the second activity but we want to receive data back from it. In this case, we will use *startActivityForResult()* instead.

Go back to *MainActivity.openSecondActivity* and replace *startActivity* with *startActivityForResult*. You will notice that a second argument *int requestCode* is needed. Write *TEXT_REQUEST* and then create a constant at the top of the class *private static final int TEXT_REQUEST = 1;*.

Next, we need to override the *startActivityForResult* method in order to process the response. Add the following method declaration in *MainActivity*:

```
@Override
protected void onActivityResult(int requestCode, int resultCode, Intent data)
{
}
```

Three arguments are needed here:

- The *requestCode* that is received from the activity; in our case we are expecting *1* (*TEXT_REQUEST*),
- The *resultCode*, that will usually be either *RESULT_OK* or *RESULT_CANCELLED*, and
- *Intent data*, which contains the data returned from the returning activity.

In the code of the method you just created, first, add the following call to *super*:

```
super.onActivityResult(requestCode, resultCode, data);
```

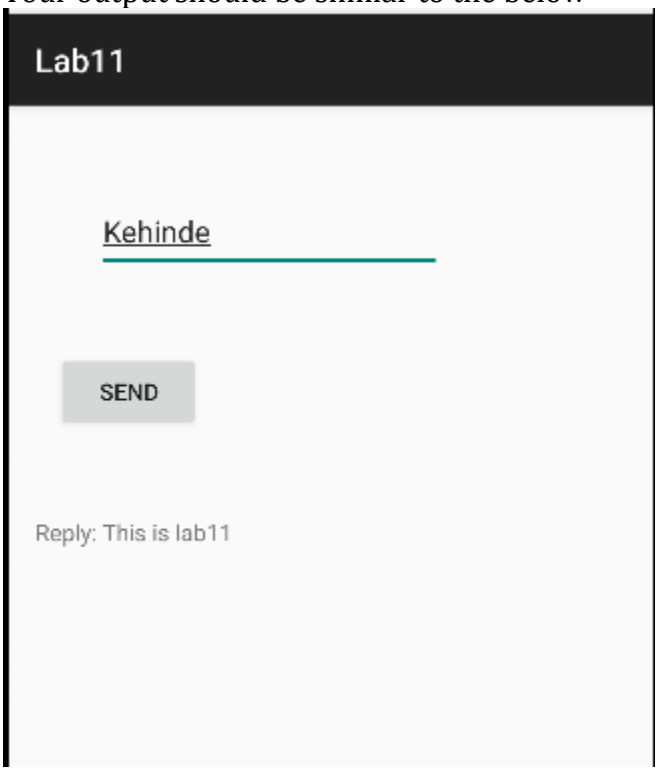
Then, check that you have received the correct *requestCode* and *resultCode*:

```
if (requestCode == TEXT_REQUEST  && resultCode  == RESULT_OK) {
}
```

And if that is the case, then get the *extra* from the intent, set it as the text of *mReplyTextView* and also set the visibility of *mReplyTextView* to *VISIBLE*. The following should be placed inside the if statement.

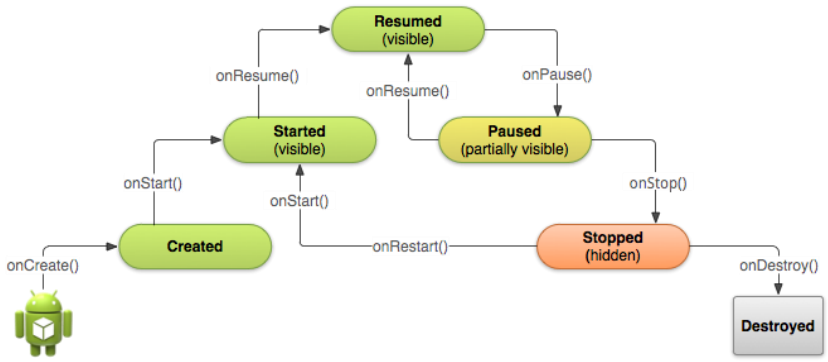
```
String reply = data.getStringExtra(SecondActivity.EXTRA_REPLY);
mReplyTextView.setText("Reply: " + reply);
mReplyTextView.setVisibility(View.VISIBLE);
```

Execute your app again and check that you can send data to and from the second activity. Your output should be similar to the below -



Activity Lifecycle

Now that we have a fully-functioning app with two activities. We will extend it to learn more about the Android *activity lifecycle*:



Android Activity Lifecycle

The activity lifecycle includes all the states in which an activity can be while the user interacts with it, and the Android system manages it. We will make use of log statements in order to track our activities' states.

Add Log Statements

Open *MainActivity.java* and add a log statement at the start of the *onCreate* method:

```
Log.d(TAG, "onCreate");
```

Add an override for the *onStart()* callback, with a corresponding log statement:

```
@Override
public void onStart(){
    super.onStart();
    Log.d(TAG, "onStart");
}
```

```
@Override
protected void onStop() {
    super.onStop();
    Log.d(TAG, "onStop");
}
```

Do the same for the *onPause()*, *onRestart()*, *onResume()*, and *onDestroy()* lifecycle callbacks. You can either copy and paste the code for *onStart*, or on Android Studio you can select *Code > Override Methods* which will display a dialog with all methods you can override in the current class. Select the five methods you want to override and click on *OK*, which will generate a template for all those methods (including the required call to the superclass). Add the appropriate log statement to each of these new methods.

Copy all of these new callbacks to *SecondActivity.java*, where you will have to add a global *TAG* variable:

```
private static final String TAG = SecondActivity.class.getSimpleName();
```

Add a log statement to the *returnReply()* method just **before** the *finish()* method call:

```
Log.d(TAG, "End Activity");
```

Run your app and interact with it (send a message to the second activity and a send a reply back) and observe the different events being printed in your **Logcat** panel.

For example, you should be able to see what happens when you click on the **Send** button on the main activity:

```
14905-14905/uk.edu.le.co2124.lab11 D/MainActivity: Button clicked
14905-14905/uk.edu.le.co2124.lab11 D/MainActivity: onPause
14905-14905/uk.edu.le.co2124.lab11 D/SecondActivity: onStart
14905-14905/uk.edu.le.co2124.lab11 D/SecondActivity: onResume
```

The main activity is first paused, then the second activity executes consecutively *onStart* and *onResume*.

Further down the log, you can find what happens when we send the reply back to the main activity:

```
14905-14905/uk.edu.le.co2124.lab11 D/SecondActivity: End Activity
14905-14905/uk.edu.le.co2124.lab11 D/SecondActivity: onPause
14905-14905/uk.edu.le.co2124.lab11 D/MainActivity: onRestart
14905-14905/uk.edu.le.co2124.lab11 D/MainActivity: onStart
14905-14905/uk.edu.le.co2124.lab11 D/MainActivity: onResume
```